

Predicción de rotación de empleados

Datos, preparación, modelos, resultados y conclusiones

Este proyecto educativo de Big Data y analítica de Recursos Humanos analiza la rotación Attrition en una empresa hipotética. Integra datos, compara clasificadores y convierte los resultados en prioridades concretas para negocio y dirección.

El objetivo es preparar información reproducible, describir patrones del conjunto y proponer áreas y medidas que un equipo de negocio pueda revisar y seguir.

Recorrido del proyecto

- Control de calidad, integración y preparación de datos con PySpark.
- Análisis exploratorio y comparación de tres modelos con scikit-learn.
- Lectura de métricas, puntuaciones y tasas por departamento.
- Comunicación de hallazgos con gráficos, panel y recomendaciones.

2. Datos, calidad y preparación

El archivo principal `data/raw/WA_Fn-UseC_-HR-Employee-Attrition.csv` contiene 1.470 registros y 35 columnas. Registra 237 casos "Yes" y 1.233 "No" (16,1 %); incluye departamento, puesto, viajes, horas extra, salario, antigüedad y otras características laborales. IBM publica un tutorial de contexto sobre clasificación de rotación.

El archivo `data/raw/encuesta_clima.csv` se genera con `scripts/generate_demo_survey.py`. Asigna cinco escalas de 1 a 5 con semilla fija 42, sin consultar Attrition, y se une al archivo principal mediante EmployeeNumber.

`scripts/check_data_quality.py` revisa presencia y nulos de la clave, duplicados, valores nulos por columna y categorías de Attrition; escribe los resultados en `/output/metrics/quality_report.json`.

Arquitectura y ETL

Docker Compose reúne Jupyter, Spark Master y PostgreSQL. Jupyter publica el puerto 8889 en la interfaz local y monta datos, scripts, notebooks y resultados. El ETL utiliza Spark en modo local (`local[*]`) y escribe en PostgreSQL cuando se invocan los scripts de persistencia.

- 1 Lee ambos CSV, normaliza textos y columnas, convierte las escalas de encuesta y elimina duplicados por clave.
- 2 Une todas las filas del archivo principal con la encuesta mediante una unión izquierda.
- 3 Crea `attrition_label` y las variables `overtime_flag`, `income_yearly` y `tenure_ratio`.
- 4 Guarda Parquet comprimido con Snappy y registra recuentos y columnas en `/output/metrics/etl_metrics.json`.

El Parquet integrado reúne 1.470 filas y 43 columnas. La memoria y el README incluyen los comandos para regenerar la encuesta, ejecutar controles y ETL y reconstruir la tabla de análisis.

3. Recorrido reproducible

Desde la raíz del repositorio, con Docker Engine y Docker Compose v2, se puede recorrer el proceso completo en este orden:

1. Arranque y acceso a Jupyter

```
docker compose up -d --build
docker compose logs jupyter
```

Abrir <http://localhost:8889/lab> e introducir el token temporal que aparece en los registros.

2. Generar encuesta, comprobar entradas y ejecutar ETL

```
docker compose run --rm jupyter python /scripts/generate_demo_survey.py --input /data/raw/WA_Fn-UseC_-HR-Employee-Attrition.csv
docker compose run --rm jupyter python /scripts/check_data_quality.py --input1 /data/raw/WA_Fn-UseC_-HR-Employee-Attrition.csv
docker compose run --rm jupyter python /scripts/etl_attrition.py --input1 /data/raw/WA_Fn-UseC_-HR-Employee-Attrition.csv --input2 /data/processed/employee_attrition.parquet
```

3. Entrenamiento y notebooks

```
docker compose run --rm jupyter python /scripts/train_ml.py --input /data/processed/employee_attrition.parquet --model all
```

En Jupyter, abrir en orden: 01_EDA_Attrition, 02_Modelado_Baseline, 03_Modelado_DL, 05_Dashboard_KPIs y 99_Informe_Final. Al terminar: `docker compose down`.

4. Análisis y modelos

01_EDA_Attrition examina estructura, distribución de Attrition y variables numéricas y categóricas. 05_Dashboard_KPIs calcula tasas generales y por departamento, puesto, horas extra, salario y antigüedad.

El pipeline excluye identificadores, constantes y columnas de etiqueta; escala variables numéricas con `StandardScaler` y codifica las categóricas con `OneHotEncoder`. El preprocesamiento se ajusta dentro de cada pliegue de validación.

Clasificadores

- Regresión logística: regularización L2, ponderación balanceada y semilla 42.
- Random Forest: 300 árboles, ponderación `balanced_subsample` y semilla 42.
- MLP de scikit-learn: capas 64 y 32, activación ReLU, parada temprana y semilla 42.

`scripts/train_ml.py --model all` calcula probabilidades OOF con validación cruzada estratificada de cinco particiones, guarda métricas y gráficos y ajusta cada pipeline con el conjunto completo para generar sus artefactos finales.

5. Resultados y visualizaciones

La tabla resume las métricas de output/metrics/model_compare.json con el mismo protocolo para los tres modelos. Valores redondeados a tres decimales:

Modelo	ROC-AUC	PR-AUC	F1 comparativo	Umbral	Exactitud a 0,5	F1 a 0,5
Regresión logística	0,819	0,552	0,533	0,732	0,753	0,479
Random Forest	0,810	0,532	0,544	0,210	0,856	0,248
MLP	0,768	0,484	0,470	0,238	0,857	0,305

Regresión Logística ofrece la mejor ordenación global según ROC-AUC y PR-AUC; Random Forest obtiene el F1 comparativo más alto. La tasa global de Attrition es 16,1 %, por lo que la exactitud se interpreta junto con las métricas de la clase positiva.

Qué muestran los gráficos

- El histograma agrupa 1.470 puntuaciones OOF de Regresión Logística en intervalos de 0,10; 773 (52,6 %) están por debajo de 0,30.
- La tasa observada es 20,6 % en Ventas (92/446), 19,0 % en Recursos Humanos (12/63) y 13,8 % en Investigación y Desarrollo (133/961).
- La tasa con horas extra es 30,5 % (127/416), frente a 10,4 % sin ellas (110/1.054).
- Entre los coeficientes de mayor magnitud aparecen representante de ventas (+1,070), viajes frecuentes (+0,856) y director/a de investigación (−1,121); sirven para orientar preguntas de negocio.

Los PNG de docs/powerbi/ resumen tasas y puntuaciones; el PBIX organiza indicadores por departamento, puesto y horas extra para consultarlos en Power BI Desktop.

6. Conclusiones y recomendaciones para dirección

La comparación ofrece dos referencias complementarias: Regresión Logística lidera ROC-AUC (0,819) y PR-AUC (0,552); Random Forest consigue el F1 más alto (0,544). En las puntuaciones OOF de Regresión Logística, el umbral comparativo 0,732 identifica 228 señales agregadas: 104 en Ventas, 115 en Investigación y Desarrollo y 9 en Recursos Humanos.

Prioridades de actuación

- Ventas:** combina la mayor tasa observada (20,6 %) con 104 señales OOF. Revisar viajes, objetivos, carga y experiencia en puestos comerciales; acordar un plan con sus responsables.
- Investigación y Desarrollo:** reúne 115 señales y 133 salidas observadas (13,8 %). Desglosar los datos por equipo y puesto para focalizar recursos donde el volumen de personas afectadas sea mayor.
- Horas extra:** la tasa de 30,5 % frente a 10,4 % orienta una revisión de turnos, carga y distribución del trabajo. Probar ajustes concretos y observar su evolución.
- Recursos Humanos:** registra 19,0 % (12/63) y 9 señales. Revisar la experiencia del equipo junto con el tamaño del área y el detalle de puestos.

Seguimiento para negocio

Presentar en el panel tasas, número de casos, señales y denominadores por departamento, puesto y horas extra. Empezar con revisiones en Ventas y en equipos con horas extra, y considerar el volumen de Investigación y Desarrollo para asignar capacidad de seguimiento. Comparar indicadores antes y después de cada acción y mantener o ampliar las medidas que mejoren de forma sostenida.

Los coeficientes de representantes de ventas y viajes frecuentes ayudan a concretar preguntas para el diagnóstico de Ventas. Las puntuaciones ordenan prioridades de revisión; la decisión sobre qué medida aplicar corresponde al análisis de cada equipo.

Apéndice. Archivos y persistencia

- `/output/metrics/model_compare.json`: métricas OOF de los modelos.
- `/output/models/`: pipelines ajustados.
- `/output/plots/`: curvas ROC y precisión-recobrado.
- `/output/bi/predictions_logreg.csv` y `feature_effects.csv`: puntuaciones y factores para los gráficos.
- `/output/metrics/etl_metrics.json` y `quality_report.json`: recuentos de ETL y controles.

`scripts/load_features_to_pg.py` carga `employees_features` en PostgreSQL; `scripts/persist_results.py` escribe las tablas `metrics` y `predictions`. La secuencia y los parámetros están documentados en el README.